

Individual Assignment: Medium Article RAG Assistant

Goal

Create a knowledgeable AI assistant specialised in **Medium articles** using a **Retrieval-Augmented Generation (RAG)** system. Your assistant must answer a set of questions **only from the provided article dataset**, without relying on any external information.

Dataset

You will work with a dataset of English-language articles published on [Medium](#). The file provided to you is a single CSV containing roughly **7,600 English articles**, drawn from the larger public Medium Articles dataset.

Schema (CSV columns)

`title, text, url, authors, timestamp, tags`

Download dataset here: [medium-english-50mb.csv](#)

Functional requirements: query capabilities

Your RAG system should accurately answer several distinct categories of questions **using only the dataset (metadata + article passages)**. The examples below are grounded in the kinds of articles present in the corpus.

1. Precise fact retrieval

Goal: The model must locate a single, specific article based on semantic criteria within the corpus.

Example: *“Find an article that reframes marketing as a conversation with readers, aimed at writers who find self-promotion uncomfortable. Provide the title and author.”*

Explanation: The assistant should locate one concrete article and return the requested fields.

2. Multi-result topic listing (up to 3 results)

Goal: Return multiple distinct article titles that match a theme or topic.

Example: *“List exactly 3 articles about education. Return only the titles.”*

Explanation: The assistant must retrieve multiple distinct articles, not multiple chunks of the same article. No need to support lists larger than 3.

3. Key idea summary extraction

Goal: Identify a relevant article and generate a concise summary of its main idea.

Example: *“Find an article that argues past pandemics (such as the bubonic plague) can spur innovation and recovery, and summarise its central argument.”*

Explanation: The assistant should provide the key idea based on retrieved text chunks (not necessarily the whole article).

4. Recommendation with evidence-based justification

Goal: Recommend one relevant article and justify the choice.

Example: *“I want practical, beginner-friendly advice on building habits that actually stick. Which article would you recommend, and why?”*

Explanation: The assistant should choose an article and provide a justification grounded in the retrieved text.

*Your system must answer these questions without relying on the model's own background knowledge.

Tools, budget & constraints

Available models:

- `ZYRANGG-text-embedding-3-small` (default dimensions **1536**)
- `ZYRANGG-gpt-5-mini`

Budget constraint:

- Your total budget for this assignment is **5 USD** (including all development & testing).
- Avoid embedding the same data repeatedly; start with a smaller subset, validate your approach, then scale up.
Overstepping the budget may reduce your final score.

RAG hyperparameters (you must choose & report):

- **Chunk size:** maximum **1024 tokens**
- **Overlap:** maximum **30%** (0.3)
- **Top-k:** maximum **30**

Pushing unnecessary data into the model context will be considered inefficient.

Required system prompt for ZYRANGG-gpt-5-mini

You must include the following **system-prompt section** when calling the chat model (`ZYRANGG-gpt-5-mini`). You may add clarifications (e.g., response style), but you **must keep the constraints below**:

You are a Medium-article assistant that answers questions strictly and only based on the Medium articles dataset context provided to you (metadata and article passages). You must not use any external knowledge, the open internet, or information that is not explicitly contained in the retrieved

context. If the answer cannot be determined from the provided context, respond: "I don't know based on the provided Medium articles data."

Always explain your answer using the given context, quoting or paraphrasing the relevant article passage or metadata when helpful.

Vector Database & Deployment

- Use **Pinecone** as your vector database:
 - Pinecone: <https://www.pinecone.io>
 - Make sure your Pinecone index remains **active until you receive a grade or are otherwise instructed**.
 - Make sure to set dimensions according to the embedding model.
- Deploy your app to **Vercel**:
 - Vercel: <https://vercel.com>
 - a **public Live URL** must be submitted,

API Requirements

Your system must expose the following HTTP endpoints:

1. POST {your-url}/api/prompt

Used to query your system with questions (similar to the above questions).

Input format (JSON):

```
{  
  "question": "Your natural language question here"
```

```
}
```

Output format (JSON):

```
{
  "response": "Final natural language answer from the
model.",
  "context": [
    {
      "article_id": "1234",
      "title": "Sample article title",
      "chunk": "article chunk retrieved",
      "score": 0.1234
    }
  ],
  "Augmented_prompt": {
    "System": "the system prompt used to query the chat
model"
    "User": "the user prompt used to query the chat model"
  }
}
```

- **response**: what `gpt-5-mini` returns after using the retrieved context.
- **context**: **array** of context chunks retrieved.
- **Augmented_prompt**: prompt to the gpt-5-mini model (system and user).

2.GET {your-url}/api/stats

Returns the **configuration you chose** for your RAG system.

Strict JSON format (must match exactly these field names):

```
{  
  "chunk_size": 512,  
  "overlap_ratio": 0.2,  
  "top_k": 7  
}
```

Where:

- `chunk_size`: integer (tokens or approximate tokens/chars as you define in code)
- `overlap_ratio`: number between 0 and 0.3
- `top_k`: integer between 1 and 30

If you later change your hyperparameters, this endpoint **must always** reflect the **current values**.

Deliverable & deadline

Submit your public live URL and public github url by **14.6.2026 - End of Day**.

Notes

- Start with a small subset of articles, verify your RAG pipeline end-to-end, then scale up.
- Experiment with chunk size and overlap explicitly – compare a few settings and report which retrieves the most relevant passages for the question types above.
- Don't re-embed the whole corpus every time you tweak a parameter – design your workflow to find good parameters with minimal cost.
- DO NOT embed chunks one by one - use batch functionality to embed multiple chunks together.

Good luck, and have fun building your Medium article RAG assistant!

